

Department of Computer Science and Engineering

Jashore University of Science and Technology

Faculty of Engineering and Technology - FET

Final Lab Report

on

CNN Image Classification with PyTorch

Date of submission: 17 August 2026

Submitted To:

Instructor: Dr. A F M Shahab Uddin
Assistant Professor

Instructor: Monishanker Halder
Assistant Professor

Instructor: Jubayer Al Mahmud
Lecturer

Department of CSE, JUST

Course Title: Artificial Intelligence and Machine Learning Lab
Course Code: CSE 3202

Submitted By:

Name: Md. Raihan Hossen
Student ID: 220105
Registration Number: 2201005
Year: 3rd
Semester: 2nd

Contents

1 Introduction **3**

1.1 How the Notebook Is Organised 3

2 The Datasets **4**

2.1 The Standard Rock Paper Scissors Dataset 4

2.2 The Custom Smartphone Dataset 4

2.3 The Domain Gap 8

3 Project Structure and Reproducibility **10**

3.1 Saving and Loading the Trained Model 11

4 Data Preprocessing **12**

4.1 Two Pipelines 12

4.2 Resize, Tensor and Normalise 12

4.3 Augmentation 13

4.4 Split and Loaders 15

5 Handling the Tensor Dimensions **17**

5.1 Channel First and the Batch Dimension 17

5.2 How the Spatial Size Shrinks 17

5.3 Flattening and the Final Scores 18

6 The Network Architecture **19**

6.1 Design 19

6.2 Building the Network, Piece by Piece 19

6.2.1 Declaring the Class 19

6.2.2 The Four Convolutional Blocks 20

6.2.3 The Shared Activation and Pooling 20

6.2.4 The Classifier Head 21

6.2.5 Passing the Image Through the Blocks 21

6.2.6 Flattening Before the Classifier 22

6.2.7 Producing the Class Scores 22

6.3 Where the Parameters Live 22

7 Training and Inference **24**

7.1 Objective and Optimiser 24

7.2 The Training Loop 25

7.2.1 Setting Up the Record of Progress 25

7.2.2 Starting Each Epoch 25

7.2.3 Learning From One Batch 25

7.2.4 Closing Out the Epoch 26

7.2.5 Measuring Without Learning 27

7.3 Running the Inference Photographs 27

7.3.1 Preparing One Photograph 27

7.3.2 Getting the Prediction 28

7.3.3 Turning Probabilities Into an Answer 28

7.3.4 Checking All Ten Automatically 29

8 Results and Evaluation **30**

8.1 Training History 30

8.2 The Standard Test Set 30

8.3 Predictions on My Own Photographs 31

8.4 Visual Error Analysis 35

9 Discussion and Conclusion **38**

Chapter 1

Introduction

Image classification is a natural fit for the Convolutional Neural Network, because a convolutional layer slides a small learnable filter across the whole image and so recognises a pattern wherever it appears, rather than learning a separate weight for every position. I chose the Rock, Paper and Scissors task because the three classes are distinct enough for a modest network to learn yet varied enough to be interesting, and because the real world testing requirement was easy to make personal: I only needed my own hand, a sheet of white paper, and my phone.

The problem is to take a photograph of a single hand forming rock, paper or scissors and output the correct class together with a confidence. What makes it more than routine is the deliberate mismatch between training and testing. The model learns only from smooth, brightly lit, computer generated renders on a white background, and is then asked to classify real phone photographs carrying uneven lighting, visible skin texture, hair and shadow. The interest lies in measuring how well knowledge learned in the clean synthetic domain transfers to the noisy real one.

My objectives were to build a complete and reproducible pipeline in PyTorch, to design a network of my own so that I would understand every layer and dimension in it, to justify each preprocessing choice rather than copy a recipe, to create a genuine custom test set from my own photographs, and to evaluate the result honestly using accuracy, class metrics, a confusion matrix and a visual inspection of both successes and failures.

1.1 How the Notebook Is Organised

The whole experiment lives in one Colab notebook, and it is worth outlining its shape before going into any single part in detail. The notebook begins by declaring a small set of constants, the image size, the batch size, the number of epochs, the learning rate and the class names, so that every later cell reads from the same source rather than repeating magic numbers. It then downloads and prepares the two datasets, defines the network, trains it or loads a saved copy, evaluates it on the synthetic test set, and finally runs it on the ten custom photographs. The chapters of this report follow that same order.

```
IMG_SIZE    = 128
BATCH_SIZE  = 64
EPOCHS      = 15
LR          = 0.001
CLASSES     = ["paper", "rock", "scissors"]
NORM_MEAN   = (0.5, 0.5, 0.5)
NORM_STD    = (0.5, 0.5, 0.5)
DEVICE      = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Chapter 2

The Datasets

2.1 The Standard Rock Paper Scissors Dataset

The training data is the public Rock, Paper and Scissors dataset, pulled automatically through Torchvision. The images are not real photographs but computer generated renders of hands against a plain white background, already sorted into folders named after their class, so the folder based loader reads the labels directly from the directory names. The data arrives as two archives, a training portion and a smaller test portion, and my notebook downloads both, unpacks them, and deletes the stray `__MACOSX` and `.DS_Store` entries that can otherwise confuse a folder based loader.

```
download_and_extract_archive(TRAIN_URL, download_root=DATA_DIR)
download_and_extract_archive(TEST_URL,  download_root=DATA_DIR)

for junk in glob.glob(f"{DATA_DIR}/**/__MACOSX", recursive=True):
    shutil.rmtree(junk, ignore_errors=True)
for junk in glob.glob(f"{DATA_DIR}/**/.DS_Store", recursive=True):
    os.remove(junk)
```

Table 2.1: Composition of the standard dataset

Portion	Paper	Rock	Scissors
Training archive (before split)	840	840	840
Test archive	124	124	124

The training archive holds two thousand five hundred and twenty images, split evenly into eight hundred and forty of each class, and the test archive holds a further three hundred and seventy two, again even at one hundred and twenty four per class. This perfect balance means plain accuracy is a fair summary rather than a misleading one.

2.2 The Custom Smartphone Dataset

The second dataset is the one I made. Following the assignment, I photographed ten gestures across the three classes, three of paper, four of rock and three of scissors, resting my hand on plain white paper to keep the background simple. Each file is named after its class and a number, for example `rock_1.jpg`, so the correct label can be recovered from the file name at test time with no separate list of answers. Table 2.2 lists the complete result.

Table 2.2: Metadata of all ten captured photographs, from a batch EXIF extraction

File	Format	Mode	Dimensions	Resolution	File size
paper_1.jpg	JPEG	RGB	3056 × 3056	9.34 MP	3.9 MB
paper_2.jpg	JPEG	RGB	3056 × 3056	9.34 MP	3.7 MB
paper_3.jpg	JPEG	RGB	3056 × 3056	9.34 MP	3.8 MB
rock_1.jpg	JPEG	RGB	3056 × 3056	9.34 MP	3.3 MB
rock_2.jpg	JPEG	RGB	3056 × 3056	9.34 MP	3.6 MB
rock_3.jpg	JPEG	RGB	3056 × 3056	9.34 MP	3.6 MB
rock_4.jpg	JPEG	RGB	3056 × 3056	9.34 MP	3.7 MB
scissors_1.jpg	JPEG	RGB	3056 × 3056	9.34 MP	3.8 MB
scissors_2.jpg	JPEG	RGB	3056 × 3056	9.34 MP	3.6 MB
scissors_3.jpg	JPEG	RGB	3056 × 3056	9.34 MP	3.4 MB

Every one of the ten files was captured on the same device, an HONOR X6c with the model code NIC LX2, on the thirteenth of August 2026, with no GPS location attached. The official model designation is normally written with a hyphen joining the two parts; it is rendered here as two words purely to keep with the hyphen free style of this report, and it refers to the same single code in every listing below. The full per image detail is listed below.

1. `paper_1.jpg`
    - Device: HONOR X6c (NIC LX2)
    - Capture date: 13 August 2026
    - File format: JPEG (JPG)
    - Image dimensions: 3056 × 3056 pixels
    - Resolution: Approximately 9.34 MP
    - Color mode: RGB, three channel color
    - File size: 3.9 MB
    - GPS data: Not available
    - Image type: Square photograph
2. `paper_2.jpg`
    - Device: HONOR X6c (NIC LX2)
    - Capture date: 13 August 2026
    - File format: JPEG (JPG)
    - Image dimensions: 3056 × 3056 pixels
    - Resolution: Approximately 9.34 MP
    - Color mode: RGB, three channel color
    - File size: 3.7 MB
    - GPS data: Not available
    - Image type: Square photograph
3. `paper_3.jpg`

- Device: HONOR X6c (NIC LX2)
- Capture date: 13 August 2026
- File format: JPEG (JPG)
- Image dimensions: 3056 × 3056 pixels
- Resolution: Approximately 9.34 MP
- Color mode: RGB, three channel color
- File size: 3.8 MB
- GPS data: Not available
- Image type: Square photograph

4. `rock_1.jpg`

    - Device: HONOR X6c (NIC LX2)
    - Capture date: 13 August 2026
    - File format: JPEG (JPG)
    - Image dimensions: 3056 × 3056 pixels
    - Resolution: Approximately 9.34 MP
    - Color mode: RGB, three channel color
    - File size: 3.3 MB
    - GPS data: Not available
    - Image type: Square photograph

5. `rock_2.jpg`

    - Device: HONOR X6c (NIC LX2)
    - Capture date: 13 August 2026
    - File format: JPEG (JPG)
    - Image dimensions: 3056 × 3056 pixels
    - Resolution: Approximately 9.34 MP
    - Color mode: RGB, three channel color
    - File size: 3.6 MB
    - GPS data: Not available
    - Image type: Square photograph

6. `rock_3.jpg`

    - Device: HONOR X6c (NIC LX2)
    - Capture date: 13 August 2026
    - File format: JPEG (JPG)
    - Image dimensions: 3056 × 3056 pixels
    - Resolution: Approximately 9.34 MP
    - Color mode: RGB, three channel color

- File size: 3.6 MB
- GPS data: Not available
- Image type: Square photograph

7. `rock_4.jpg`
    - Device: HONOR X6c (NIC LX2)
    - Capture date: 13 August 2026
    - File format: JPEG (JPG)
    - Image dimensions: 3056×3056 pixels
    - Resolution: Approximately 9.34 MP
    - Color mode: RGB, three channel color
    - File size: 3.7 MB
    - GPS data: Not available
    - Image type: Square photograph
8. `scissors_1.jpg`
    - Device: HONOR X6c (NIC LX2)
    - Capture date: 13 August 2026
    - File format: JPEG (JPG)
    - Image dimensions: 3056×3056 pixels
    - Resolution: Approximately 9.34 MP
    - Color mode: RGB, three channel color
    - File size: 3.8 MB
    - GPS data: Not available
    - Image type: Square photograph
9. `scissors_2.jpg`
    - Device: HONOR X6c (NIC LX2)
    - Capture date: 13 August 2026
    - File format: JPEG (JPG)
    - Image dimensions: 3056×3056 pixels
    - Resolution: Approximately 9.34 MP
    - Color mode: RGB, three channel color
    - File size: 3.6 MB
    - GPS data: Not available
    - Image type: Square photograph
10. `scissors_3.jpg`
    - Device: HONOR X6c (NIC LX2)
    - Capture date: 13 August 2026

- File format: JPEG (JPG)
- Image dimensions: 3056×3056 pixels
- Resolution: Approximately 9.34 MP
- Color mode: RGB, three channel color
- File size: 3.4 MB
- GPS data: Not available
- Image type: Square photograph

Three properties mattered later for the pipeline. The images are square at three thousand and fifty six pixels a side, an aspect ratio of one to one, which meant that resizing to a square target would not stretch the hand. Every file is three channel RGB colour, matching the training data, which settled the decision to keep the whole pipeline in colour. And each file is a compact three to four megabyte JPEG, a reminder that file size and pixel count are different things: a photograph of only a few megabytes expands into more than nine million pixels once it is opened.



(a) Paper (b) Rock (c) Scissors

Figure 2.1: Three of the ten custom photographs, one per class, showing the plain background and the real hand texture the synthetic data lacks.

2.3 The Domain Gap

Before modelling I checked my early impression that the two kinds of image differ, by measuring average brightness on a scale from zero for black to one for white.

```
def mean_brightness(path):
    arr = np.asarray(Image.open(path).convert("RGB"), dtype=np.float32) / 255.0
    return arr.mean()
```

Across my photographs the mean brightness was about zero point five eight, ranging from roughly zero point five three to zero point six one, noticeably darker and narrower than the brilliant synthetic renders. Figure 2.2 plots the brightness I measured for each of my photographs, together with the typical brightness band of the synthetic renders, and the two do not overlap. Plotting both groups together, they formed two separate clusters rather than one, which is the visual signature of a domain gap. This told me that if I did nothing to prepare the model for the difference, it would likely stumble on my darker photographs, and it is what motivated the colour and brightness augmentation described next.

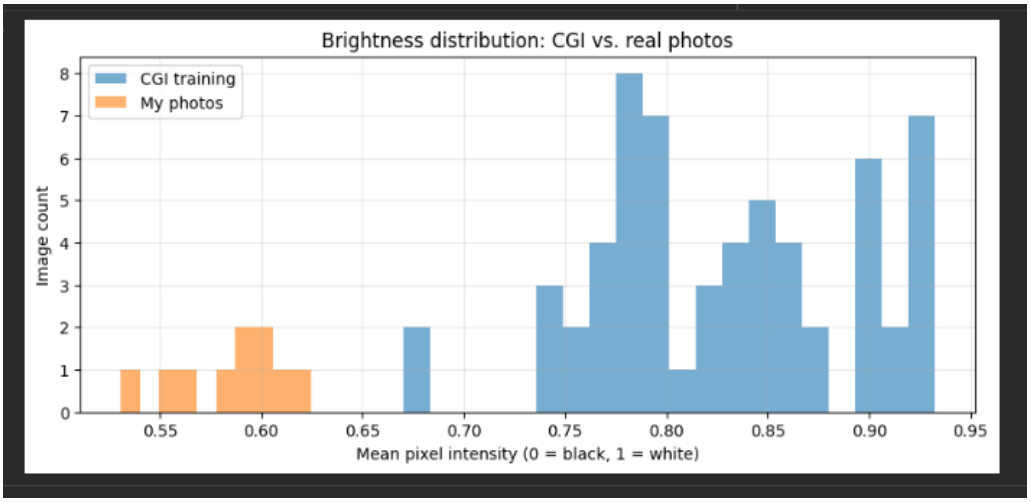


Figure 2.2: Mean brightness of each custom photograph, on a scale from zero for black to one for white, against the typical brightness band of the synthetic renders. My photographs sit in a lower, narrower band, which is the domain gap made visible.

Chapter 3

Project Structure and Reproducibility

A firm requirement of this assignment is that the whole experiment reproduces itself with no manual steps. Everything the project needs lives in one public repository, laid out simply as shown in Figure 3.1.

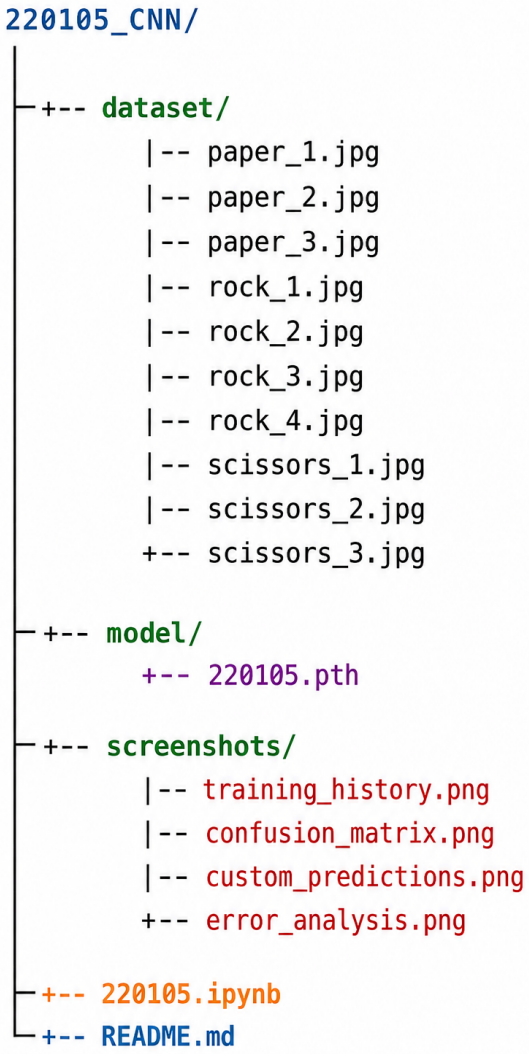


Figure 3.1: The project repository. The custom photographs sit in the dataset folder, the trained weights and history in the model folder, the README figures in the screenshots folder, and the notebook and README at the top level.

Both data sources are fetched automatically. The standard images are downloaded through Torchvision, and the custom photographs are obtained by cloning the repository from inside the

notebook.

```
!rm -rf {REPO_DIR}
!git clone {REPO_URL}
```

3.1 Saving and Loading the Trained Model

The trained network is saved as a state dictionary, PyTorch's standard mapping from each layer name to its learned numbers. I inspected the saved file: it is about nine and a third megabytes and holds the weights and biases of every layer plus the batch normalisation parameters and their running statistics. Those running statistics explain a small discrepancy in the counts. The learnable parameters, which gradient descent adjusts, number two million three hundred and thirty nine thousand seven hundred and fifteen, while the saved file contains seven hundred and eight more; that difference is exactly the running averages the four batch normalisation layers keep, which are saved so the model behaves identically when reloaded but are not trained.

```
if os.path.exists(MODEL_PATH):
    model.load_state_dict(torch.load(MODEL_PATH, map_location=DEVICE))
    with open(HISTORY_PATH) as f:
        history = json.load(f)
    print("Loaded saved weights, skipping training.")
else:
    history = train(model, train_loader, val_loader, EPOCHS)
    torch.save(model.state_dict(), MODEL_PATH)
    with open(HISTORY_PATH, "w") as f:
        json.dump(history, f)
```

Because the weights are committed to the repository, the notebook offers two honest paths through the same code. If the saved weights are present it loads them and skips training, reproducing every figure in seconds; if they are absent it trains from scratch and saves them. The reader gets a correct result either way, and the expensive training is only paid for when needed.

Chapter 4

Data Preprocessing

Preprocessing is the bridge between a folder of image files and a network that expects tensors of a fixed shape and scale, and its choices shaped the outcome as much as the architecture did. A network cannot consume a JPEG directly: it needs every input at the same spatial size, its pixels as floating point numbers in a small range centred on zero, and, during training, enough variety that it learns the general idea of each gesture rather than memorising specific images.

4.1 Two Pipelines

The project needs two pipelines, and confusing them is a classic mistake. The training pipeline includes random augmentation for variety. The evaluation pipeline, used for the validation set, the test set and my custom photographs, must be strictly deterministic, because when measuring performance I want every image processed identically with no randomness clouding the result.

```
eval_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=NORM_MEAN, std=NORM_STD)
])
```

```
train_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(20),
    transforms.ColorJitter(brightness=0.4, contrast=0.3,
                           saturation=0.3, hue=0.05),
    transforms.ToTensor(),
    transforms.Normalize(mean=NORM_MEAN, std=NORM_STD)
])
```

4.2 Resize, Tensor and Normalise

The first step resizes every image to a square of one hundred and twenty eight pixels a side. A fixed size is forced by the architecture, because the first fully connected layer expects a fixed number of inputs. The particular value is a tradeoff: larger keeps more detail but costs more memory and time, smaller is cheap but discards detail. One hundred and twenty eight balances the two, and it is a power of two, so the four halvings the network performs give sixty four, thirty two, sixteen and eight with nothing lost to rounding. Because my photographs were already square, this resize is a pure reduction in scale with no stretching.

The tensor conversion does two things: it turns the whole number pixels from zero to two hundred and fifty five into floating point numbers scaled to the range zero to one, and it reorders the

axes from the height, width, channel layout images normally use into the channel first layout PyTorch expects.

Normalisation then shifts and rescales the values, per channel, following

$$\text{normalised} = \frac{\text{pixel} - \text{mean}}{\text{standard deviation}}. \tag{4.1}$$

With a mean and standard deviation of one half, a pixel of zero becomes negative one, a pixel of one becomes positive one, and a middling half becomes zero, so the range zero to one is stretched onto the symmetric range negative one to positive one, centred on zero. Networks train more stably when their inputs are centred and small in scale, because the gradients are better behaved, which is why centring on zero is one of the oldest pieces of practical advice in the field.

To confirm this with my own eyes rather than on faith, I wrote a small helper that reverses the normalisation and the axis order, so a transformed tensor can be displayed as an ordinary image again.

```
def denormalize(tensor):
    t = tensor.clone().cpu()
    for c in range(3):
        t[c] = t[c] * NORM_STD[c] + NORM_MEAN[c]
    return t.clamp(0, 1).permute(1, 2, 0).numpy()
```

4.3 Augmentation

The augmentation transforms appear only in training and are my main defence against overfitting and my main tool for narrowing the domain gap. The horizontal flip mirrors the image with probability one half, which is safe because a gesture is just as valid mirrored. The rotation turns the image by up to twenty degrees either way, because my photographs are never perfectly upright and I want the network to ignore small tilts. Figure 4.1 shows one of my photographs after several draws of the flip and rotation alone.



Figure 4.1: The same photograph after several independent draws of the random horizontal flip and the random rotation. The hand is mirrored in some draws and tilted by a different small angle in each, while remaining clearly the same gesture.

A quick check that convinced me it was working: I passed one training image through the training pipeline eleven times and displayed the results side by side. Every one was recognisably the same hand, yet every one differed, some flipped, some tilted, some brighter or cooler. The

network would never see the same exact image twice, which is exactly the point.

```
sample_img = Image.open(sample_path).convert("RGB")
fig, axes = plt.subplots(1, 6, figsize=(15, 3))
for ax in axes:
    augmented = train_transform(sample_img)
    ax.imshow(denormalize(augmented))
    ax.axis("off")
```

4.4 Split and Loaders

The dataset provides training and test portions but no validation portion, so I split the training portion, keeping four fifths for training and one fifth for validation, and left the test portion untouched for a single final measurement. Reusing the test set during development would quietly inflate the final number.

Table 4.1: How the images were divided

Role	Images	Source
Training	2016	80 percent of the training archive
Validation	504	20 percent of the training archive
Test	372	the separate test archive, untouched

A folder based dataset carries a single transform for every image, but I needed the training and validation slices of the same dataset to use different transforms, augmentation for one and plain processing for the other. I gave each slice its own copy of the dataset with the right transform attached. Without this the validation set would have been secretly augmented and its measurements untrustworthy.

```
full_train = ImageFolder(TRAIN_DIR)
n_val   = int(0.2 * len(full_train))
n_train = len(full_train) - n_val

train_subset, val_subset = random_split(full_train, [n_train, n_val])

train_set = copy.copy(train_subset)
train_set.dataset = copy.copy(full_train)
train_set.dataset.transform = train_transform

val_set = copy.copy(val_subset)
val_set.dataset = copy.copy(full_train)
val_set.dataset.transform = eval_transform
```

Finally, the data loaders feed images in batches of sixty four, which is faster and gives a smoother training signal than one image at a time.

```
train_loader = DataLoader(train_set, batch_size=BATCH_SIZE,
                          shuffle=True,  num_workers=2)
val_loader   = DataLoader(val_set,   batch_size=BATCH_SIZE,
                          shuffle=False, num_workers=2)
test_loader  = DataLoader(test_set,  batch_size=BATCH_SIZE,
                          shuffle=False, num_workers=2)
```

The training loader shuffles before every epoch so the network cannot learn anything from the order; the validation and test loaders do not shuffle, so the evaluation is perfectly repeatable. A batch drawn from the training loader had the shape sixty four, three, one hundred and twenty eight, one hundred and twenty eight, with pixel values from about negative one to positive one, confirming the whole pipeline worked.

Chapter 5

Handling the Tensor Dimensions

A tensor flowing through the network constantly changes shape, and if the shape at any point does not match what the next layer expects, the whole thing stops with an error. Understanding the shapes is also understanding the network, because they record what each layer does. I write every shape in the order batch, channel, height, width.

5.1 Channel First and the Batch Dimension

The tensor conversion silently reorders an image from the height, width, channel layout it is stored in into the channel first layout PyTorch expects, where the second dimension is always the channel. Every convolutional layer reads its input assuming this, so an image left in the other layout would be misread from the first operation.

During training and testing the images carry a leading batch dimension, the number of images processed together, which simply rides along unchanged through every layer while the other three dimensions transform. This becomes important at inference on a single photograph, which has only three dimensions, channel, height and width, and which the network refuses because it always expects a leading batch dimension. The fix is to insert a batch dimension of size one, turning a shape of three, one hundred and twenty eight, one hundred and twenty eight into one, three, one hundred and twenty eight, one hundred and twenty eight, so a single image looks like a batch of one.

```
single = eval_transform(img)          # shape: (3, 128, 128)
batch  = single.unsqueeze(0)          # shape: (1, 3, 128, 128)
```

5.2 How the Spatial Size Shrinks

Through the four convolutional blocks two movements happen at once: the channel count grows as each convolution learns more features, while the height and width shrink as each pooling step halves them. The convolutions themselves preserve the spatial size, because each uses a three by three filter with a one pixel border of padding chosen precisely so the output matches the input; all the shrinking is done by pooling. Starting from one hundred and twenty eight, four halvings give sixty four, thirty two, sixteen and eight, all whole numbers. Table 5.1 traces the full shape for one image.

Table 5.1: The shape at every stage of the network, for one input image

Stage	Operation	Output shape (channel, height, width)
Input	the preprocessed image	`(3, 128, 128)`
Block one	convolution then pooling	`(32, 64, 64)`
Block two	convolution then pooling	`(64, 32, 32)`
Block three	convolution then pooling	`(128, 16, 16)`
Block four	convolution then pooling	`(128, 8, 8)`
Flatten	unroll into a single vector	`(8192)`
Classifier one	reduce to 256 values	`(256)`
Classifier two	reduce to 3 class scores	`(3)`

I also printed the shape after every layer directly from the notebook while debugging, which is what let me build the table above from real output rather than from arithmetic alone.

```
x = images
print("input   ", x.shape)
x = pool(relu(bn1(conv1(x)))); print("block 1 ", x.shape)
x = pool(relu(bn2(conv2(x)))); print("block 2 ", x.shape)
x = pool(relu(bn3(conv3(x)))); print("block 3 ", x.shape)
x = pool(relu(bn4(conv4(x)))); print("block 4 ", x.shape)
x = x.view(x.size(0), -1);     print("flatten ", x.shape)
```

5.3 Flattening and the Final Scores

The transition to the classifier is where dimensions are handled most explicitly. After the fourth block the data is one hundred and twenty eight channels of eight by eight, and the classifier expects a single flat vector. Its length is the product of those three dimensions, one hundred and twenty eight times eight times eight, which is eight thousand one hundred and ninety two. This number is not free; it is fixed by the input size and the number of pooling steps, and it is exactly why the first fully connected layer is built to expect eight thousand one hundred and ninety two inputs. The flattening line keeps the batch dimension and merges the rest:

```
x = x.view(x.size(0), -1)
```

The classifier then reduces eight thousand one hundred and ninety two values to two hundred and fifty six and finally to three raw class scores. A softmax is applied across the class dimension to turn these into three probabilities that sum to one, and specifying that dimension correctly is what keeps each image's probabilities separate rather than mixing scores between images.

```
probs = torch.softmax(logits, dim=1)
```

Chapter 6

The Network Architecture

6.1 Design

The network follows the classic pattern of four convolutional blocks feeding a small classifier. Each block has the same structure: a convolution, then batch normalisation, then a rectified linear activation, then pooling. The convolutions expand the channels from three to thirty two, then sixty four, then one hundred and twenty eight, then one hundred and twenty eight, reflecting the idea that as the spatial size shrinks and each channel represents a larger, more abstract piece of the image, more channels are useful. Batch normalisation after each convolution keeps the signal at a stable scale and makes training faster and steadier. The rectified linear activation, which passes positive values and zeroes negatives, provides the nonlinearity without which the whole network would collapse into a single linear operation. Pooling keeps the strongest response in each two by two neighbourhood, which both halves the size and gives a little tolerance to small shifts in position.

After the blocks the data is flattened and passes through a dropout step and two fully connected layers, reducing eight thousand one hundred and ninety two values to two hundred and fifty six and then to three. The dropout, which randomly switches off half of its inputs during training, is the main explicit defence against overfitting and is placed just before the classifier, because as Table 6.1 shows that is where almost all the parameters live.

6.2 Building the Network, Piece by Piece

Rather than present the whole class at once, I walk through it in the order I wrote it, because each piece exists for a reason that is easier to see on its own.

6.2.1 Declaring the Class

Everything starts by declaring my network as a class that inherits from the base module PyTorch provides for all networks. Inheriting from it is what gives my network the ability to keep track of its own layers, move itself onto the graphics processor, and save and load its weights, none of which I have to write myself. The number of classes is set to three by default, because my task has exactly three gestures, but leaving it as an argument means the same class could be reused for a different number of classes without editing it. The call to the parent constructor on the second line is required boilerplate: it lets the base module set up its internal bookkeeping before I start adding layers of my own.

```
1 class CNN(nn.Module):
2     def __init__(self, num_classes=3):
3         super(CNN, self).__init__()
```

6.2.2 The Four Convolutional Blocks

These eight lines create the feature extractor of my network, and they are the part that actually looks at the image. Each pair creates one convolution together with its matching batch normalisation.

Reading the convolution lines from left to right, the first number is how many channels come in and the second is how many go out. So `conv1` accepts the three colour channels of my photograph and produces thirty two feature channels, `conv2` takes those thirty two and produces sixty four, `conv3` takes sixty four and produces one hundred and twenty eight, and `conv4` keeps it at one hundred and twenty eight. This growth is deliberate: as the image shrinks and each remaining cell stands for a larger region of the original hand, I want more channels available to describe the increasingly complex patterns being detected.

The kernel size of three means each filter examines a three by three patch at a time, which is small enough to detect local details like the edge of a finger. The padding of one adds a single pixel border around the image before the filter slides across it, and I chose this specifically so the convolution does not shrink the image at all. That keeps the size arithmetic clean, because it means every reduction in size in my network comes from pooling alone and nowhere else.

Each batch normalisation layer is told how many channels it will receive, and that number always matches whatever its paired convolution just produced. Its job is to keep the numbers flowing through my network at a steady scale, which stopped my training from being sensitive to the starting conditions and let it settle quickly.

```
        self.conv1 = nn.Conv2d(3,   32,  kernel_size=3, padding=1)
        self.bn1   = nn.BatchNorm2d(32)

        self.conv2 = nn.Conv2d(32,  64,  kernel_size=3, padding=1)
        self.bn2   = nn.BatchNorm2d(64)

        self.conv3 = nn.Conv2d(64,  128, kernel_size=3, padding=1)
        self.bn3   = nn.BatchNorm2d(128)

        self.conv4 = nn.Conv2d(128, 128, kernel_size=3, padding=1)
        self.bn4   = nn.BatchNorm2d(128)
```

6.2.3 The Shared Activation and Pooling

These two lines create the activation and the pooling step, and it is worth noticing that I create only one of each even though my network uses them four times over. That is possible because neither of them learns anything; they have no weights of their own, so the same object can be reused after every block with no difference in behaviour. Creating four copies would have worked equally well but would only have added clutter.

The activation passes positive values through unchanged and turns negative values into zero. Without something like this between my convolutions, stacking four of them would have been pointless, because a chain of purely linear operations collapses into a single linear operation and could never separate the three gestures.

The pooling layer takes every two by two square of cells and keeps only the largest value from it. The kernel size of two sets the size of that square and the stride of two makes it move two cells at a time, so the squares never overlap and the height and width are exactly halved. This

is what takes my image from one hundred and twenty eight pixels a side down to eight over the four blocks, and keeping only the strongest response in each little neighbourhood also makes my network slightly forgiving about a feature shifting by a pixel or two, which matters when a hand is never photographed in exactly the same position twice.

```
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
```

6.2.4 The Classifier Head

These three lines create the part of my network that turns the extracted features into an actual decision.

The dropout layer will randomly switch off half of the values passing through it during training, choosing a different half every time. The figure of nought point five is that fraction. I placed it here, immediately before the fully connected layers, because those layers are where almost all of my parameters sit and therefore where memorising the training data was most likely to happen.

The first fully connected layer is where the input size of my images becomes locked to the architecture. The expression one hundred and twenty eight times eight times eight is written out rather than as the number eight thousand one hundred and ninety two on purpose, because it shows exactly where the number comes from: one hundred and twenty eight channels, each eight cells tall and eight cells wide, which is what my data has become after the four blocks. This layer compresses all of that down to two hundred and fifty six values. The second fully connected layer then takes those two hundred and fifty six values and reduces them to three, one score for each of paper, rock and scissors.

```
        self.dropout = nn.Dropout(0.5)
        self.fc1 = nn.Linear(128 * 8 * 8, 256)
        self.fc2 = nn.Linear(256, num_classes)
```

6.2.5 Passing the Image Through the Blocks

Everything above only built the layers; nothing has processed an image yet. The forward method is where an actual batch of photographs travels through my network, and these four lines are the feature extraction stage.

Each line performs one complete block, and the nesting reads from the inside outwards. Taking the first line as the example, the convolution runs first on the incoming data, its output is handed to batch normalisation, that result is handed to the activation, and finally that is handed to pooling. So one line covers convolution, normalisation, activation and pooling in that exact order. The result is stored back into the same variable, which is then fed into the next line, and this is what chains the four blocks together into a sequence.

Following the shape as it moves down these four lines is the clearest picture of what my network does. My image enters as three channels of one hundred and twenty eight by one hundred and twenty eight. After the first line it is thirty two channels of sixty four by sixty four. After the second it is sixty four channels of thirty two by thirty two. After the third it is one hundred and twenty eight channels of sixteen by sixteen. After the fourth it is one hundred and twenty eight channels of eight by eight. The picture keeps getting smaller while the description of it keeps getting richer.

```
    def forward(self, x):
        x = self.pool(self.relu(self.bn1(self.conv1(x))))
        x = self.pool(self.relu(self.bn2(self.conv2(x))))
        x = self.pool(self.relu(self.bn3(self.conv3(x))))
        x = self.pool(self.relu(self.bn4(self.conv4(x))))
```

6.2.6 Flattening Before the Classifier

This single line is the bridge between the two halves of my network, and it caused me the most trouble of anything in the architecture. The convolutional half works on a three dimensional block of channels, height and width, but the fully connected half only understands a single flat list of numbers, so the block has to be unrolled.

The first argument keeps the batch dimension exactly as it is, so the images being processed together stay separate and never have their values mixed. The second argument, written as negative one, tells PyTorch to work out the remaining length by itself, which it does by dividing the total number of values by the batch size. For my network that comes to eight thousand one hundred and ninety two per image. This is the same number written into the first fully connected layer earlier, and the two must agree exactly; when I first got it wrong, this was precisely where my network refused to run.

```
        x = x.view(x.size(0), -1)
```

6.2.7 Producing the Class Scores

These four lines finish the job. Dropout runs first, randomly zeroing half of the flattened values while my network is training, and doing nothing at all once the model is switched to evaluation mode for testing my photographs.

The next line pushes what remains through the first fully connected layer and immediately applies the activation to its output, squeezing eight thousand one hundred and ninety two values down to two hundred and fifty six nonlinear ones. The line after that passes those through the second fully connected layer to give three numbers, one per class.

Notice that no activation is applied after this last layer. That is deliberate, because the three numbers returned here are raw scores rather than probabilities, and the cross entropy objective I train with expects to receive exactly these raw scores and performs the conversion internally. When I want readable probabilities for my custom photographs, I apply the softmax myself outside the model, which is what produces the confidence percentages reported later.

```
        x = self.dropout(x)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

6.3 Where the Parameters Live

Having built the network, I wanted to know how its size was distributed, so I counted the parameters layer by layer rather than only as a single total.

The first line here asks my model for every parameter it owns, keeps only the trainable ones, counts the individual numbers inside each, and adds them up. The loop below does the same thing but reports it per layer, printing each layer's name alongside its shape and its own count, and it is what produced the breakdown in Table 6.1.

```
total = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Total learnable parameters: {total:,}")

for name, p in model.named_parameters():
    print(name, tuple(p.shape), p.numel())
```

Table 6.1: Learnable parameters by layer

Layer	**Parameters**	**Share**
Convolution blocks one to four (with normalisation)	241,536	10.32%
Classifier layer one	2,097,408	89.64%
Classifier layer two	771	0.03%
Total learnable	**2,339,715**	**100%**

The striking fact is that the first fully connected layer holds nearly ninety percent of all the parameters, because it connects every one of the eight thousand one hundred and ninety two flattened values to each of its two hundred and fifty six outputs. All four convolutional blocks together hold only about ten percent, because a convolutional filter is small and reused across the whole image. This is why placing dropout right before that layer is the right decision, and it is a concrete illustration of why convolutional layers are considered so efficient.

Chapter 7

Training and Inference

7.1 Objective and Optimiser

Training minimises the cross entropy objective, which grows when the network is confidently wrong and shrinks when it places most of its probability on the correct class. The implementation I used accepts the raw class scores directly and converts them internally, so the network only needs to output raw scores. The optimiser is Adam, a forgiving form of gradient descent that adapts the step size per parameter and converges quickly without much tuning. Table 7.1 lists the settings.

Table 7.1: Training configuration

Setting	**Value**
Input image size	128×128 pixels
Batch size	64
Epochs	15
Learning rate	0.001
Objective	cross entropy
Optimiser	Adam
Device	graphics processor when available, else processor

These three lines set up everything training needs before the loop can start.

The first line creates an actual instance of the network class I designed in the previous chapter, telling it that my task has three classes, and then immediately moves it onto whichever device the notebook selected. That move matters, because the model and the images it processes must live in the same place; if the model sat on the graphics processor while my batches sat on the ordinary processor, every forward pass would fail.

The second line creates the objective. This is the single number training will try to make smaller. It compares the three raw scores my network produces against the true label and grows large when my network is confidently wrong. A convenient detail is that this particular implementation applies the softmax internally, which is exactly why my network's final layer returns raw scores with no activation.

The third line creates the optimiser and hands it two things: every parameter in my network, which is what it is allowed to adjust, and the learning rate, which controls how large each adjustment is. Passing `model.parameters()` is what connects the optimiser to my specific network, so that when it takes a step it is stepping on my convolutions and my classifier and nothing else.

```
model     = CNN(num_classes=3).to(DEVICE)
```

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR)
```

7.2 The Training Loop

7.2.1 Setting Up the Record of Progress

The function takes my model, the two loaders and the number of epochs, and begins by creating a dictionary with four empty lists. These will collect the loss and accuracy on both the training and validation sets, one entry appended per epoch. I keep this record because it is what I later save to disk and what the training curves in the results chapter are drawn from, so I never have to retrain just to redraw a plot.

```
def train(model, train_loader, val_loader, epochs):
    history = {"train_loss": [], "train_acc": [],
               "val_loss": [], "val_acc": []}
```

7.2.2 Starting Each Epoch

The outer loop runs once per epoch, fifteen times in my case, where each epoch is one full pass over all two thousand and sixteen training images.

The call to `model.train()` does not train anything despite its name; it simply puts my network into training mode. That flag is what tells my dropout layer to actually start switching values off and my batch normalisation layers to use the statistics of the current batch. Setting this at the top of every epoch matters because the validation pass at the end of the epoch will have switched the model into the other mode, and without this line my second and later epochs would train with dropout silently disabled.

The three running totals are then reset to zero. They must start fresh each epoch, because I want each epoch's reported numbers to describe that epoch alone rather than accumulate across the whole run.

```
    for epoch in range(epochs):
        model.train()
        running_loss, correct, total = 0.0, 0, 0
```

7.2.3 Learning From One Batch

This inner loop is where my network actually learns. Each time round, the loader hands over sixty four preprocessed images together with their true labels, already resized, converted to tensors, normalised and randomly augmented by the training pipeline. The first line inside moves both onto the same device as my model.

The next five lines are the heart of the whole project. Clearing the gradients first is essential, because PyTorch adds new gradients onto whatever is already stored rather than replacing them, so without this line my network would be updated using a confused mixture of the current batch and every batch before it. Then the batch is pushed through my network to produce three raw scores per image, and the objective compares those scores against the true labels to give a single

number describing how wrong my network currently is. The backward call works backwards through every layer of my network to compute how that error would change if each individual parameter were nudged. Finally the optimiser applies those nudges, adjusting all two million odd parameters slightly in the direction that reduces the error.

The last three lines only keep score and have no effect on learning. The loss is multiplied by the batch size before being added, because the objective returns an average over the batch and I want a true total to divide correctly at the end. The accuracy line takes the highest of the three scores as my network's prediction, compares it against the truth, and counts how many were right.

```
        for images, labels in train_loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * images.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
            total += labels.size(0)
```

7.2.4 Closing Out the Epoch

Once every batch has been seen, the running totals are turned into averages by dividing by the number of images processed. Then my model is run over the validation set through the evaluation function described below, which reports how it performs on five hundred and four images it did not learn from during this epoch.

Watching the training and validation numbers side by side is the single most useful signal I had while developing. If the training accuracy had climbed while the validation accuracy stalled, that would have told me my network was memorising rather than learning, and I would have needed stronger augmentation or more dropout. In practice the two stayed close, which is what gave me confidence the model was genuinely generalising within the synthetic domain.

All four numbers are appended to the history lists and printed so I can watch progress live in the notebook rather than staring at a blank cell for several minutes. After the final epoch the completed history is returned to the caller, which saves it alongside the weights.

```
        train_loss = running_loss / total
        train_acc  = correct / total

        val_loss, val_acc = evaluate(model, val_loader)

        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)

        print(f"Epoch {epoch+1}/{epochs}  "
              f"train_loss={train_loss:.4f} train_acc={train_acc:.4f}  "
              f"val_loss={val_loss:.4f} val_acc={val_acc:.4f}")

```

```
    return history
```

7.2.5 Measuring Without Learning

I wrote this evaluation function once and used it for two different jobs: the validation check after every epoch, and the final measurement on the three hundred and seventy two image test set. Only the loader passed in changes.

Two things make this function measure rather than learn. The call to `model.eval()` switches my network into evaluation mode, which turns dropout off completely so the full classifier is used, and tells batch normalisation to use the running averages it accumulated during training rather than recomputing statistics from whatever batch happens to be in front of it. The block that disables gradients then stops PyTorch from building the machinery it would need to work backwards, which is pure waste here since nothing is being adjusted, and it makes the pass noticeably faster and lighter on memory.

Inside, the loop mirrors the training loop but with the four learning lines removed. Images are moved to the device, pushed through my network, and compared against the truth, with the loss and correct counts accumulated exactly as before. The function returns the average loss and the accuracy as a pair, which is why the training loop can unpack it into two variables in one line.

```
def evaluate(model, loader):
    model.eval()
    running_loss, correct, total = 0.0, 0, 0

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item() * images.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
            total += labels.size(0)

    return running_loss / total, correct / total
```

The notebook uses a graphics processor when one is available and falls back to the processor otherwise, moving both the network and every batch onto the chosen device so they always meet in the same place. When training finishes, the weights and the epoch by epoch history are saved, which is what lets the experiment reload and reproduce quickly.

7.3 Running the Inference Photographs

7.3.1 Preparing One Photograph

This is the part of the project the whole pipeline was built for: taking one photograph of my own hand and getting an answer out of it.

The first line opens the file and immediately forces it to three channel colour. That conversion

is a safeguard I added deliberately after being caught out by it, because a file that arrives as grayscale or with a transparency layer would have the wrong number of channels and would fail at my very first convolution, which expects exactly three. Forcing the conversion at the moment of opening means whatever the file happened to contain, what leaves this line always has three channels.

The second line applies the evaluation pipeline, the deterministic one, with no augmentation whatsoever. This is the most important rule of the whole inference stage: the transforms here must match those used during training, including the exact same normalisation constants. My nine megapixel photograph is resized to one hundred and twenty eight by one hundred and twenty eight, converted to a tensor with its axes reordered to channel first, and normalised onto the same range from negative one to positive one that my network learned on. Had I skipped the normalisation, my network would be seeing values on a completely different scale from anything in its training and the prediction would be meaningless.

The third line does two small but necessary things. My photograph at this point has only three dimensions, channel, height and width, but my network always expects a leading batch dimension because it was built to process many images at once. Adding a dimension of size one at the front makes my single photograph look like a batch that happens to contain exactly one image. It is then moved onto the same device as the model.

```
img = Image.open(path).convert("RGB")
tensor = eval_transform(img)
tensor = tensor.unsqueeze(0).to(DEVICE)
```

7.3.2 Getting the Prediction

Switching to evaluation mode here is not optional, and forgetting it is a subtle mistake because the network still runs and still returns numbers, they are simply quietly wrong. In evaluation mode my dropout is fully off so the entire classifier contributes, and my batch normalisation uses its stored running averages rather than trying to compute statistics from a batch of one, which would be meaningless. Disabling gradients again saves time and memory since no learning happens here.

The model returns three raw scores for my photograph. The softmax then converts them into three probabilities that add up to one, and the dimension argument tells it to normalise across the three classes rather than across the batch. The index at the end selects the first, and only, image out of the batch of one I created earlier, leaving me with three plain probabilities.

```
model.eval()
with torch.no_grad():
    logits = model(tensor)
    probs  = torch.softmax(logits, dim=1)[0]
```

7.3.3 Turning Probabilities Into an Answer

These three lines convert the probabilities into something readable. The first finds the position of the largest probability, which is my network's decision. The second uses that position to look up the matching name in my class list, turning a bare index into the word paper, rock or scissors. The third takes the probability at that same position and multiplies by one hundred to give the

confidence percentage, which is what appears above each hand in the prediction figures later, such as the ninety three point two percent on the one photograph my network got wrong.

```
pred_idx   = probs.argmax().item()
pred_cls   = CLASSES[pred_idx]
confidence = probs[pred_idx].item() * 100
```

7.3.4 Checking All Ten Automatically

Finally this loop applies everything above to my whole custom set. It gathers every photograph in the dataset folder and sorts them so the order is predictable rather than dependent on how the file system happens to list them.

For each one, the true label is recovered straight from the file name by taking the part before the underscore. This is the payoff of the naming scheme I chose back when capturing the photographs: because a file called `rock_1.jpg` carries its own answer, I never had to write or maintain a separate list of correct labels that could fall out of step with the images.

The prediction and confidence are obtained, and all four pieces are stored together. The last two lines count how many predictions matched their true label and print the score, which is what produced the result of nine correct out of ten.

```
results = []
for path in sorted(glob.glob(f"{CUSTOM_DIR}/*.jpg")):
    true_cls = true_label_from_filename(path)
    pred_cls, confidence = predict(path)
    results.append((os.path.basename(path), true_cls, pred_cls, confidence))

correct = sum(1 for _, t, p, _ in results if t == p)
print(f"Custom photo accuracy: {correct}/{len(results)}")
```

Chapter 8

Results and Evaluation

8.1 Training History

Figure 8.1 shows the loss and accuracy over fifteen epochs. The loss falls steeply then flattens, and the accuracy climbs steeply then settles into a high plateau. By the end the final training accuracy was about ninety nine percent and the final validation accuracy reached one hundred percent, with the lowest validation loss around the twelfth epoch. The closeness of the training and validation curves is important, because a large gap would signal overfitting; its absence shows the network learned features that transferred to unseen images, at least within the synthetic domain.

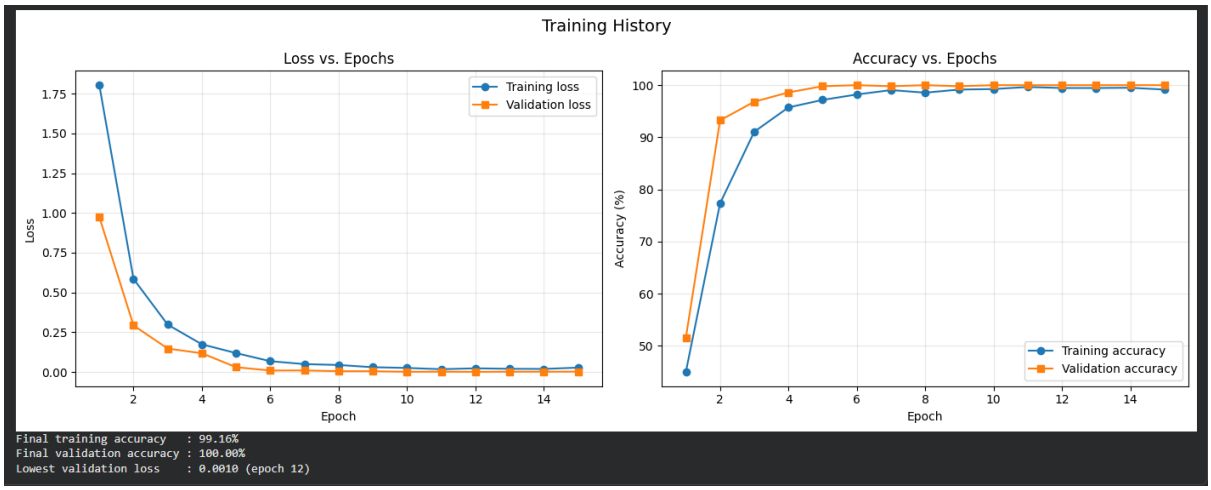


Figure 8.1: Loss and accuracy over fifteen epochs for the training and validation sets. The steep early improvement and stable plateau indicate clean convergence.

I was careful not to over read the perfect validation score, because the validation images come from the same synthetic source as the training images. A perfect score there shows mastery of that kind of image but says nothing directly about real photographs, which is exactly why the project insists on a separate custom test set.

8.2 The Standard Test Set

On the untouched test set of three hundred and seventy two synthetic images the network reached ninety two point two zero percent accuracy, correctly classifying three hundred and forty three. The overall figure hides a clear variation between classes, shown in Table 8.1: the network was flawless on rock, strong on scissors, but noticeably weaker on paper. This pattern recurs on my own photographs, which suggests it reflects the genuine relative difficulty of the gestures.

```
from sklearn.metrics import classification_report, confusion_matrix

report = classification_report(y_true, y_pred, target_names=CLASSES,
                               digits=4, output_dict=True)
cm = confusion_matrix(y_true, y_pred)
```

Table 8.1: Performance on the standard test set, by class

Class	Accuracy	Precision	Recall	F score
Paper	83.06%	0.9717	0.8306	0.8957
Rock	100.00%	0.8732	1.0000	0.9323
Scissors	93.55%	0.9355	0.9355	0.9355

The confusion matrix in Figure 8.2 shows where the errors fall. The rock row is perfectly clean and the scissors row nearly so, but of the one hundred and twenty four true papers, one hundred and three were correct while thirteen were mistaken for rock and eight for scissors. So the single largest source of error is paper being read as rock, which is interesting because to a human a flat hand and a fist look nothing alike.

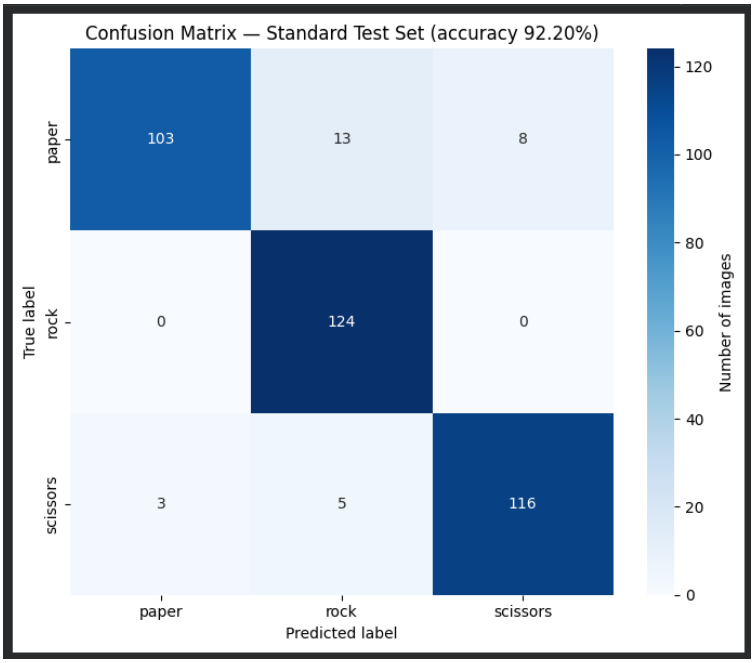


Figure 8.2: The confusion matrix on the standard test set. Errors concentrate in the paper row, where thirteen papers were read as rock and eight as scissors.

8.3 Predictions on My Own Photographs

The real test is running the network on the ten photographs of my own hand. Figures 8.3, 8.4 and 8.5 show the predictions for all ten, with the predicted class and confidence that the model returned above each hand. The network correctly classified nine of the ten, ninety percent, and did so with high confidence on almost all, frequently close to one hundred percent. Given

that it had never seen a single real photograph in training, only smooth synthetic renders, correctly recognising nine of ten real photographs is an excellent outcome, and it suggests the augmentation strategy of jittering brightness and colour to stretch the training images toward real conditions paid off.



Figure 8.3: Predictions on four of my ten custom photographs, all correctly classified.

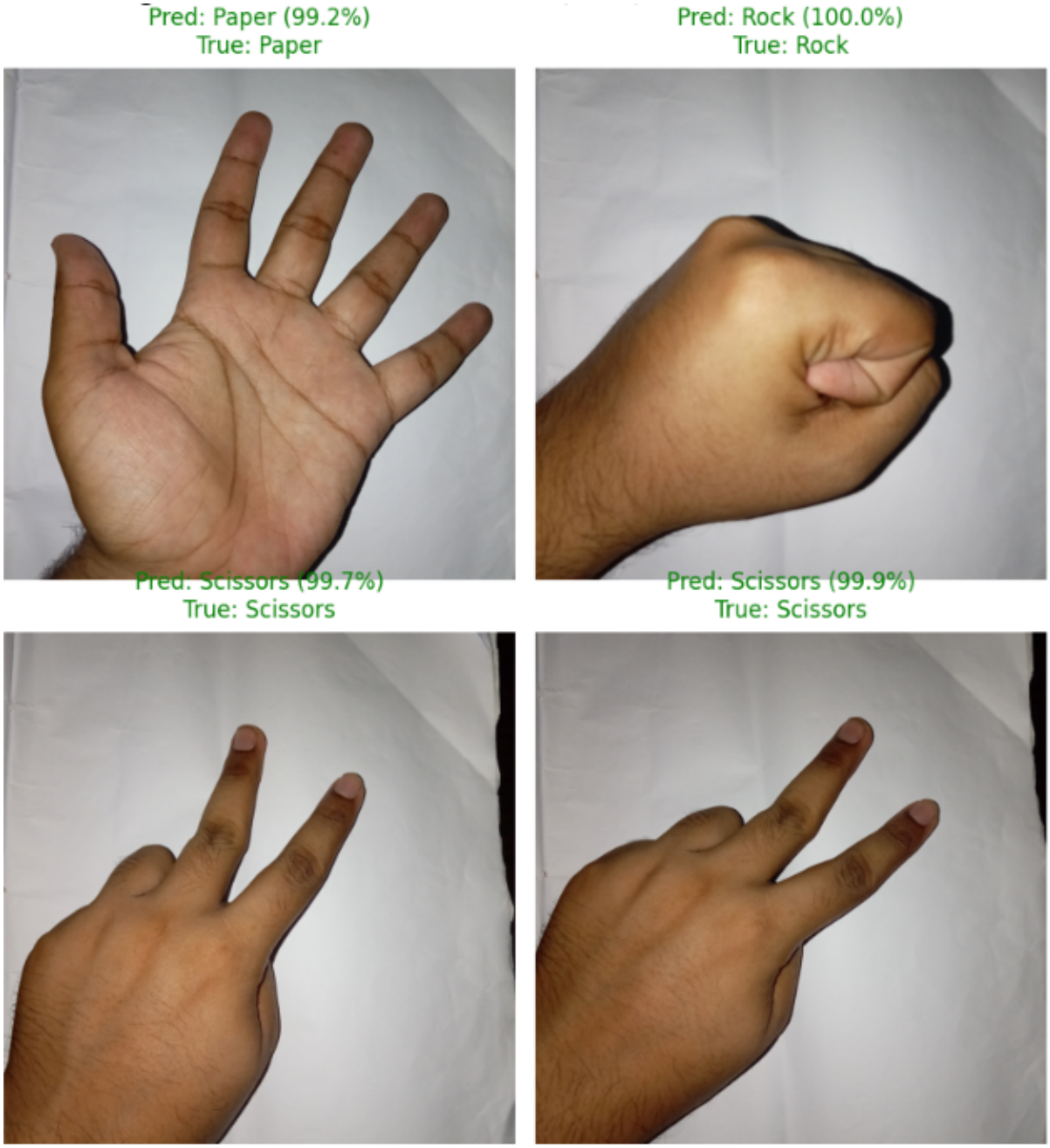


Figure 8.4: Predictions on four more of my ten custom photographs, all correctly classified.

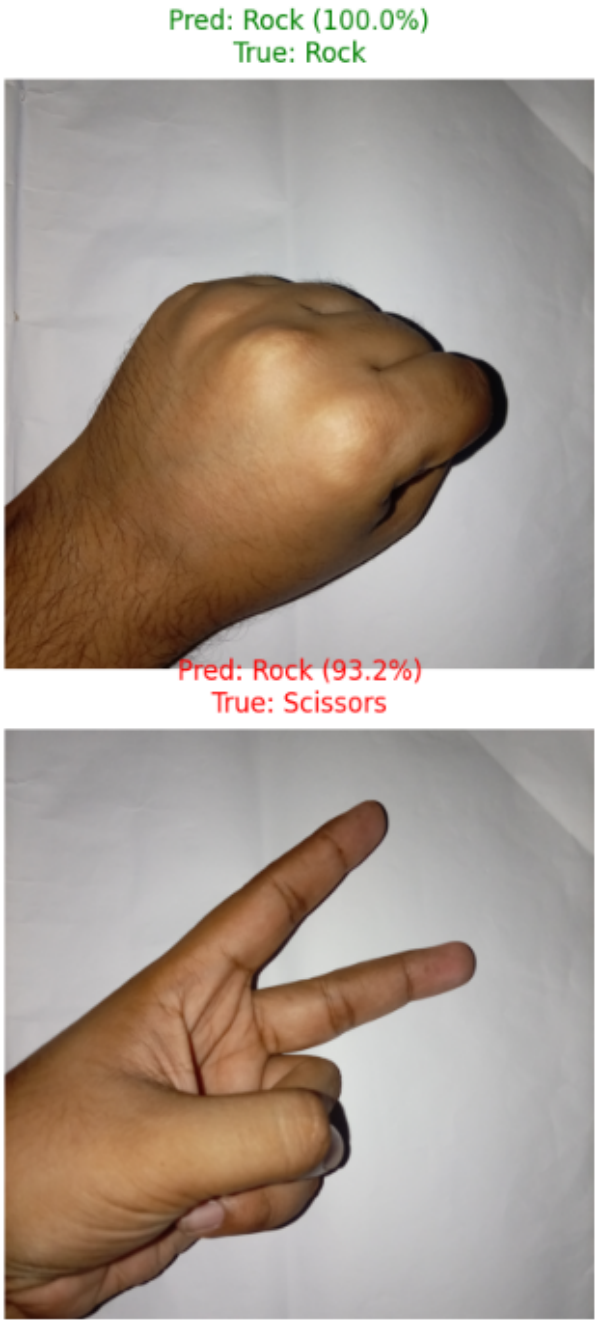


Figure 8.5: Predictions on two of my ten custom photographs, one wrongly classified., `scissors_3.jpg`, is the single misclassification, predicted as rock with 93.2 percent confidence.

The one failure, `scissors_3.jpg` misread as rock, is worth understanding. Scissors is formed by extending two fingers while the rest of the hand stays curled, and from certain angles the extended fingers can be less distinct against the folded hand, so the silhouette takes on some of the compact quality of a fist. Given the network already leaned toward the rock label, a slightly ambiguous scissors tipping into rock is a reasonable error rather than a wild one.

8.4 Visual Error Analysis

To understand the errors on the synthetic test set more concretely, I displayed three of the misclassified images individually, each with its true and predicted label. Looking at the images behind the error counts, rather than only the numbers, is what let me actually explain the mistakes rather than just report them.

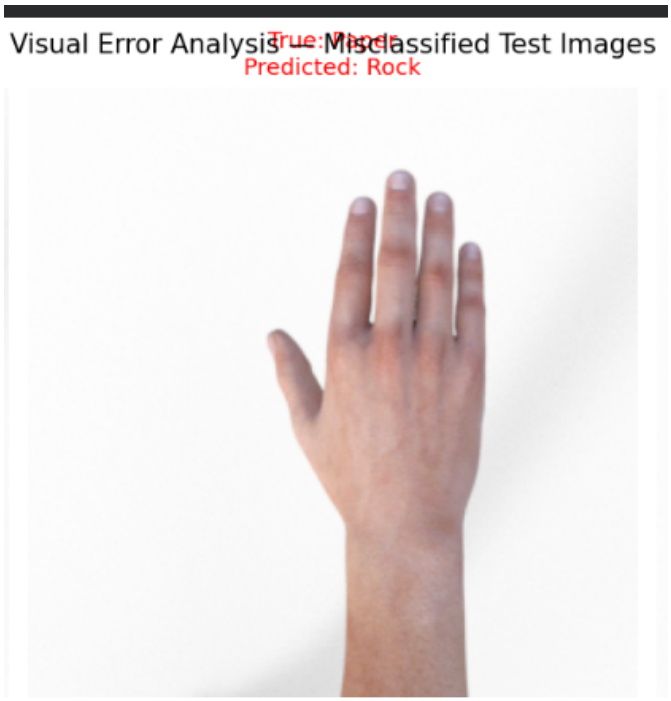


Figure 8.6: True label: paper. Predicted label: rock. The fingers are close together rather than spread, so the outline of the hand reads as a more compact shape.

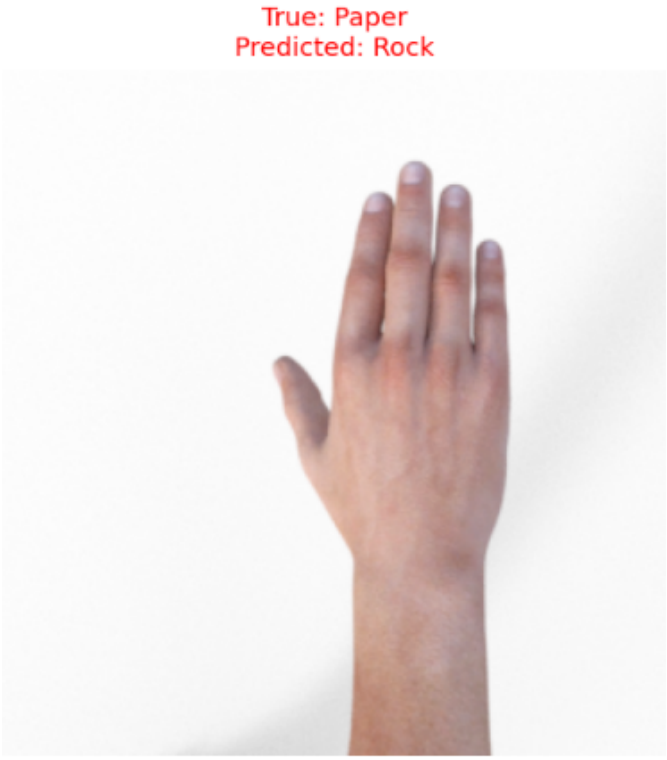


Figure 8.7: True label: paper. Predicted label: rock. As in the previous image, the fingers are not properly spread out, which is why the model read this open hand as a fist.



Figure 8.8: True label: paper. Predicted label: scissors. Here the hand is angled so that some fingers are hidden behind others, leaving a silhouette with roughly two visible finger like shapes, close to what the network associates with scissors.

These three images were paper in every case, but the network read the first two as rock because the fingers were not properly spread out, leaving a compact silhouette closer to a fist. In the third the hand is angled outward with some fingers hidden behind others, and the two or so visible finger shapes are what pushed the prediction toward scissors instead. In every case the misclassification is understandable once the image itself is examined, which is reassuring: the network is failing on genuinely ambiguous poses rather than on clear, typical ones.

Chapter 9

Discussion and Conclusion

The theme running through the project was the gap between the synthetic world the network learned in and the real world it was tested in. I measured that gap before modelling and found it real, then chose augmentation specifically to bridge it, and the network crossed it successfully most of the time. Had I trained on the pristine renders alone, I suspect it would have been far more brittle on my darker, more textured photographs. The gap was not eliminated, one photograph still fell through, but it was substantially bridged, and doing so deliberately rather than by luck was the most instructive part of the work.

Paper was the hardest class, most often confused with rock, because an open hand varies far more in appearance than a fist: seen at an angle that hides the spread of the fingers, a foreshortened open hand presents a surprisingly compact silhouette that drifts toward the shape of a fist. Rock, by contrast, was classified flawlessly, because a closed fist looks much the same from every angle and shares little of its shape with the other two gestures. Consistency, not intricacy, is what makes a class easy for a network that keys on overall shape.

Several practical problems shaped the final pipeline. An image without three channels failed at the first layer until I converted every image to colour at the point of loading. A folder based dataset carries one transform, which would have forced the validation set to share the training augmentation until I gave each split its own copy. A loose reasoning about shapes led to a mismatch at the classifier until I traced the dimensions layer by layer, exactly as printed in the debugging snippet in Chapter Five. And the downloaded archives carried stray files that a folder based loader can misread until I deleted them. None was dramatic, but each, left unsolved, would have quietly broken or degraded the pipeline.

The project has clear limitations. Ten photographs are enough to demonstrate real world behaviour qualitatively but far too few for a confident statistical claim, and a single different photograph would swing the figure by ten percentage points. The training data, though large, is synthetic and uniform, and augmentation could stretch its appearance but not add information the renders never contained. The network is small by modern standards and holds most of its parameters in one fully connected layer. Natural extensions would be a larger and more varied set of real photographs, training that incorporates real images, and a more efficient or pretrained architecture.

In summary, the finished notebook retrieves its data automatically, preprocesses it through a reasoned pipeline, trains a network I designed and understand in full, and reports a thorough set of results with no manual intervention. It reached ninety two point two zero percent on the synthetic test set and correctly classified nine of my ten real photographs. More than the numbers, the project taught me that preprocessing and augmentation matter as much as architecture, that tracking tensor dimensions explicitly is the surest defence against structural errors, and that looking closely at both the data and the mistakes, rather than only the headline accuracy, is where the real understanding lies.